# Lista de Exercícios: Estruturas Híbridas

AEDS 2 · PUC Minas · baseada no Tópico 6 (Estruturas Híbridas) e no Tópico 3 (Inserção em C/C++)

**Como usar esta lista**

- As questões vão do aquecimento (contas de custo) até a implementação no estilo da prova, e terminam com questões para "abrir a cabeça".
- Em **toda** questão de implementação, dê a complexidade em Θ do **melhor e do pior caso** e diga qual hipótese você usou (distribuição uniforme? árvores balanceadas?).
- Antes de codar, desenhe a estrutura e escreva o algoritmo em português. Só depois passe para Java (ou C, quando pedido).
- Se a questão não disser nada, suponha árvores binárias de pesquisa **sem elementos repetidos** e listas **com nó cabeça**.

DO SLIDE exercício do professor ESTILO PROVA no formato das provas antigas
ABRE A CABEÇA exige pensar além do slide

**Lembretes rápidos**

- Custo de uma estrutura híbrida: $\mathbf{C = C_1(k) + C_2(n_i)}$.
- Árvore balanceada com m nós, no pior caso: ⌊lg m⌋ + 1 visitas. Lista com m elementos: pior m, médio (m+1)/2.
- Para ⌊lg m⌋, ache a maior potência de 2 que é ≤ m.

## Parte A · Contas de custo

**A1.** DO SLIDE · EXERCÍCIO 4

Uma agenda tem n = 5.200 nomes distribuídos uniformemente entre as 26 letras.

1. Calcule o número máximo de nós/células visitados na pesquisa pelo nome em: (a) lista única, (b) árvore de listas, (c) árvore de árvores, (d) árvore única balanceada e (e) vetor de árvores.
2. Refaça os itens (b) e (c) acima supondo que as letras foram inseridas em **ordem alfabética**.
3. Qual o custo de pesquisar pelo CPF na árvore de listas?

*Tente sem olhar a resolução do slide e depois confira.*

**A2.** ESTILO PROVA

Repita a A1 (itens a até e) para **n = 52.000** e k = 26. Monte uma tabela com o cálculo e o resultado de cada estrutura. Qual estrutura ganha? Por quanto?

**A3.** ABRE A CABEÇA

Uma agenda brasileira tem n = 26.000 nomes, mas **metade deles começa com M** (muitas Marias!). Os outros 13.000 se dividem igualmente entre as 25 letras restantes. As letras estão na árvore balanceada do slide (o M é a raiz).

a. Calcule o pior caso da pesquisa pelo nome na **árvore de listas**. Cuidado: o pior caso agora depende da letra. Compare "nomes com M" com "nomes de uma letra no último nível".
b. Calcule o pior caso na **árvore de árvores** (todas as árvores do 2º nível balanceadas).
c. O resultado de (b) te surpreendeu? Explique por que a árvore de árvores "sentiu" muito menos a má distribuição do que a árvore de listas.

**A4.** ABRE A CABEÇA

No slide, k = 26 é uma constante, e por isso a árvore de listas continua Θ(n). Agora imagine uma estrutura em que o número de grupos **cresce com n**: k = √n grupos, com √n elementos em cada um (1º nível: árvore balanceada; 2º nível: lista).

a. Escreva o custo de pesquisa no pior caso em função de n.
b. Qual a ordem de complexidade? Ela é melhor ou pior que Θ(n)? E que Θ(lg n)?
c. Por que "dividir por 26" não muda a ordem, mas "dividir por √n" muda?

**A5.** ESTILO PROVA

Na árvore de árvores com k letras e $n_i$ = n/k, considere que o 1º nível está balanceado, mas **todas as árvores do 2º nível degeneraram** (as palavras foram inseridas em ordem alfabética). Dê o custo de pesquisa no pior caso em função de n e k, e a ordem de complexidade. A árvore de árvores ainda vale a pena em relação à árvore única degenerada?

## Parte B · Rastreio e desenho

**B1.** ESTILO PROVA

Use o método `construir(esq, dir)` do slide para montar uma árvore só com as letras de **'A' a 'J'**.

a. Escreva a ordem em que as letras são inseridas (mostre o cálculo do `meio` em cada chamada).
b. Desenhe a árvore resultante.
c. Calcule o pior caso e o caso médio de pesquisa de uma letra (use a tabela "nível × nós").

**B2.** ESTILO PROVA

Numa árvore de árvores (1º nível com as 26 letras, como no slide), insira na árvore da letra T, nesta ordem:

`TATU, TELA, TIGRE, TOCA, TALCO, TUBO, TEIA`

a. Desenhe a árvore do T depois das inserções.
b. Quantos nós são visitados (1º + 2º nível) em `pesquisar("TOCA")`? E em `pesquisar("TREM")`, que não existe?
c. Em que ordem o `mostrar('T')` (caminhamento central) imprime as palavras?

**B3.** ESTILO PROVA

Considere a matriz de listas 2 × 2 abaixo (cada célula aponta para uma lista com nó cabeça):

```
        coluna 0                coluna 1
linha 0 [a00] → 5 → 8 → 13      [a01] → 2
linha 1 [a10] → (vazia)         [a11] → 7 → 13 → 1
```

a. Quantas células de lista são visitadas em `pesquisar(13)`, percorrendo a matriz linha por linha e parando no primeiro encontro? E em `pesquisar(9)`?

b. Quantas em `pesquisar(1, 1, 13)`, que só olha a célula da linha 1, coluna 1?

c. O que `mostrar()` imprime se ele percorrer a matriz coluna por coluna?

**B4.** ESTILO PROVA (ligação com o Tópico 3)

Na árvore de árvores em C, o 2º nível usa `void inserirRec(char* s, No** i)`. Com a árvore do T contendo só TATU (raiz) e TELA, faça o quadro "chamada / i / *i / decisão" da inserção de **TIGRE**. Invente os endereços dos nós e diga de qual campo é o endereço guardado em `i` na última chamada.

## Parte C · Implementação

### Classes da árvore de listas (use nas questões C1, C4 e C5)

```
class Contato { String nome, telefone, email; int cpf; }

class Celula {
   Contato contato; Celula prox;
   Celula() { this(null); }                       // usado para o nó cabeça
   Celula(Contato c) { contato = c; prox = null; }
}

class No {
   char letra; No esq, dir;
   Celula primeiro, ultimo;                       // lista COM nó cabeça
   No(char l) { letra = l; esq = dir = null; primeiro = ultimo = new Celula(); }
}

class Agenda { No raiz; /* construtor insere 'A'..'Z' balanceado */ }
```

**C1.** DO SLIDE · EXERCÍCIO 1

Crie as classes acima e, na classe `Agenda`, os métodos `inserir(Contato)`, `remover(String nome)`, `pesquisar(String nome)` e `pesquisar(int cpf)`. No construtor, insira as 26 letras maiúsculas (sem acento) de forma balanceada. Para cada método, mostre o melhor e o pior caso em função de n.

*Atenção no `remover`: o que acontece com o `ultimo` se você remover justamente a última célula?*

### C2. DO SLIDE · EXERCÍCIO 2

Na árvore de árvores (`class No { String elemento; No esq, dir; }` e `class NoLetra { char letra; NoLetra esq, dir; No raiz; }`), implemente:

a. `boolean pesquisar(String)`, `void inserir(String)` e `void mostrar()`.
b. `boolean hasStringTam10()`: existe alguma String de tamanho 10?
c. `boolean hasStringTam10(char c)`, em duas versões: Strings de tamanho 10 que **começam** com c e Strings de tamanho 10 que **terminam** com c.

Para cada método, dê o custo no pior caso em função de n, k e $n_i$, e explique por que as duas versões do item (c) têm ordens diferentes.

### C3. DO SLIDE · EXERCÍCIO 3

Implemente `pesquisar(int elemento)`, `pesquisar(int i, int j, int elemento)`, `inserir(int i, int j, int elemento)` e `mostrar()` para a matriz de listas abaixo. Dê o custo de cada método em função de l (linhas), c (colunas) e $n_{ij}$ (elementos da lista da célula ij).

```
class Celula { int elemento; Celula prox; }
class CelulaMatriz {
   CelulaMatriz sup, inf, esq, dir;              // vizinhos na matriz
   Celula primeiro, ultimo;                      // lista COM nó cabeça
}
class MatrizDeListas { CelulaMatriz inicio; int linha, coluna; }
```

### C4. ESTILO PROVA

Na árvore de listas, implemente:

a. `int contar(char c)`: retorna quantos contatos começam com a letra c.
b. `char letraMaisPopular()`: retorna a letra com mais contatos (em caso de empate, qualquer uma).

Dê a complexidade em Θ dos dois. O item (b) consegue ser mais rápido que Θ(n) sem mudar os atributos? E se você **pudesse** mudar os atributos da classe `No`, o que acrescentaria para que (a) ficasse Θ(1) depois de achar a letra?

### C5. ESTILO PROVA ABRE A CABEÇA

Implemente `Celula unificar()` na árvore de listas: o método deve juntar **todas** as listas numa única lista simples, com as letras em ordem alfabética (todos os A, depois todos os B, ...), e retornar o início dela.

a. Primeira versão: crie células novas para a lista unificada. Qual a complexidade?
b. Segunda versão: **não crie nenhuma célula**, apenas religue os ponteiros das listas que já existem (as listas originais podem ser destruídas). Qual a complexidade agora? Dica: cada lista guarda o `primeiro` **e** o `ultimo`.
c. Qual caminhamento na árvore de letras garante a ordem alfabética?

### C6. ESTILO PROVA

Na árvore de árvores, implemente `String ultimaPalavra()`, que retorna a última String em ordem alfabética de todo o dicionário (ou `null` se ele estiver vazio).
*Cuidado: a letra mais à direita do 1º nível (Z) pode ter uma árvore vazia. O que fazer nesse caso?* Dê a complexidade no melhor e no pior caso.

**C7.** ESTILO PROVA (em C, ligação com o Tópico 3)
Implemente em C a inserção na árvore de árvores, usando `No**` no 2º nível:

```
typedef struct No { char* elemento; struct No *esq, *dir; } No;
typedef struct NoLetra { char letra; struct NoLetra *esq, *dir; No* raiz; } NoLetra;
NoLetra* raiz;                                  // árvore de letras já construída

void inserir(char* s);                          // pública
static void inserirRec(char* s, No** i);        // recursiva com No**
```

Use `strcmp(a, b)`, que funciona como o `compareTo` do Java (negativo, zero ou positivo). Em seguida, reescreva `inserirRec` em C++ com `No*& i` e aponte as diferenças.

**C8.** ESTILO PROVA ABRE A CABEÇA
**Lista de árvores.** Uma escola guarda suas t turmas numa lista simples (ordenada pelo número da turma). Cada célula da lista aponta para uma árvore binária de pesquisa com as matrículas dos alunos da turma.

```
class No { int matricula; No esq, dir; }
class CelulaTurma { int turma; No raiz; CelulaTurma prox; }
class Escola { CelulaTurma primeiro; }          // lista SEM nó cabeça
```

a. Implemente `boolean pesquisar(int turma, int matricula)`.
b. Implemente `int pesquisar(int matricula)`, que retorna a turma do aluno (ou -1).
c. Dê o custo dos dois em função de t e $n_i$. Compare com a árvore de árvores: o que mudou por o 1º nível ser uma lista e não uma árvore?

**C9.** ESTILO PROVA ABRE A CABEÇA
**Árvore de filas.** Um hospital organiza o atendimento com uma árvore binária de setores (chave: código do setor). Cada setor tem uma **fila flexível** de senhas.

```
class Celula { int senha; Celula prox; }
class NoSetor {
   int codigo; NoSetor esq, dir;
   Celula primeiro, ultimo;                     // fila COM nó cabeça
   int tamanho;                                 // quantas senhas na fila
}
class Hospital { NoSetor raiz; }
```

a. `void chegar(int setor, int senha)`: coloca a senha no fim da fila do setor.
b. `int atender(int setor)`: remove e retorna a primeira senha da fila do setor (lance exceção se estiver vazia). Não esqueça de atualizar o `tamanho` e o `ultimo`, quando necessário.
c. `int setorMaisCheio()`: retorna o código do setor com a maior fila.
d. Dê a complexidade de cada método em função de s (número de setores). Por que o atributo `tamanho` é importante para o item (c)?

## Parte D · Para abrir a cabeça

**D1.** ABRE A CABEÇA

Verdadeiro ou falso? **Justifique cada item** (na prova, a justificativa vale mais que o V/F).

a. Uma árvore de listas com k = 26 letras balanceadas tem pesquisa pelo nome Θ(lg n).

b. O vetor de árvores é sempre melhor que a árvore de árvores.

c. Trocar o 1º nível da árvore de árvores (árvore de letras) por um vetor muda a ordem de complexidade da pesquisa.

d. Na árvore de árvores, procurar palavras que **terminam** com a letra c é Θ(lg n).

e. Se todas as árvores do 2º nível degenerarem, a árvore de árvores fica Θ(n), mesmo com o 1º nível balanceado.

f. Inserir na árvore de listas é Θ(1) porque cada lista tem um ponteiro para o último.

g. Numa árvore de listas, mostrar todos os contatos em ordem alfabética **completa** (e não só pela primeira letra) custa o mesmo que mostrar todos eles em qualquer ordem.

**D2.** ABRE A CABEÇA

O escritório de advocacia agora quer pesquisar rápido **tanto pelo nome quanto pelo CPF**.

a. Por que a árvore de listas sozinha não resolve?

b. Proponha uma solução usando **duas estruturas** que apontam para os **mesmos** objetos `Contato` (sem duplicar os dados). Desenhe.

c. Qual o custo de inserir e de remover um contato na sua solução? O que pode dar errado se você esquecer de atualizar uma das duas estruturas?

**D3.** ABRE A CABEÇA

Em vez de agrupar pela primeira letra, um colega propõe agrupar pelas **duas primeiras letras** ("MA", "MB", ..., "ZZ"), ou seja, k = 26 × 26 = 676 grupos, numa árvore de listas.

a. Quanto custa o 1º nível balanceado no pior caso?

b. Com n = 676.000 distribuídos uniformemente, compare o pior caso dessa estrutura com o da árvore de listas por 1 letra.

c. Os nomes reais **não** se distribuem de forma uniforme pelas duas primeiras letras (pense em "MA" × "XQ"). Isso ajuda ou atrapalha a ideia? Que estrutura do 2º nível você escolheria para se proteger disso?

**D4.** ABRE A CABEÇA

**Três níveis.** Imagine uma árvore de letras (1ª letra) em que cada nó aponta para outra árvore de letras (2ª letra), e cada nó desta aponta para uma lista de nomes.

a. Generalize a fórmula do slide: escreva o custo C da pesquisa como soma de três parcelas.

b. Com n = 676.000 uniformes, calcule o pior caso.

c. Quantos nós existem no 2º nível, no total? Essa memória extra é um preço razoável?

**D5.** ABRE A CABEÇA

Quando a **árvore de listas** é uma escolha melhor que a árvore de árvores? Pense em pelo menos duas situações (dicas: custo de inserção, ordem em que os contatos foram cadastrados, quantidade de nomes por letra).

**Gabarito**

Esta lista vem sem gabarito de propósito, para você tentar primeiro. Quando resolver (no caderno ou no código), me manda as respostas que eu corrijo apontando o que está certo, o que pode melhorar e por quê, mantendo o seu jeito de escrever o código.